Construction d'un arbre hiérarchique hybride pour la génération de code frontend

Dossier détaillé du cheminement méthodologique

Avertissement méthodologique

Ce document retrace l'intégralité du raisonnement suivi jusqu'à ce jour. Les décisions architecturales (choix hybride, ordre géométrie-sens, structure du nœud) sont présentées comme **arrêtées et justifiées**. En revanche, la combinaison algorithmique **Gestalt + XY-Cut** est actuellement **à l'essai** : elle repose sur un raisonnement solide et documenté, mais **aucun résultat expérimental ne vient encore la valider**. Ce point est signalé explicitement à chaque section concernée.

Contents

1 Contexte et objectif du mémoire **2**

2 Panorama complet des méthodes existantes **2**
- 2.1 Les familles identifiées . . . 2
- 2.2 Pourquoi une approche hybride mathématique + MLLM, et pas une autre famille 2

3 L'ordre d'exécution : géométrie d'abord, sens ensuite **3**
- 3.1 Preuve vérifiée : l'ordre de DesignCoder . . . 3
- 3.2 Preuve vérifiée : l'ordre de ScreenCoder . . . 3
- 3.3 Notre choix : l'inverse, assumé . . . 3

4 Les huit champs du nœud **4**

5 Nuance sur le vide de recherche identifié **4**
- 5.1 VSA (Visual-Structural Alignment), décembre 2025 . . . 4
- 5.2 ReDraw (Moran et al.) . . . 5
- 5.3 Reformulation de notre contribution . . . 5

6 Justification du terrain web **5**

7 Le choix du détecteur **5**
- 7.1 Candidats écartés . . . 5
- 7.2 Détecteur retenu . . . 6

8 État d'avancement pratique — résultats non encore validés **6**
- 8.1 Installation et déploiement . . . 6
- 8.2 Corrections nécessaires . . . 6
- 8.3 Tests menés . . . 6
- 8.4 Limite non résolue à ce jour . . . 6
- 8.5 Prochaines étapes immédiates . . . 7

9 Feuille de route générale **7**

10 Bibliographie **7**

1 Contexte et objectif du mémoire

Le pipeline général de génération de code frontend à partir d'une maquette suit toujours la même chaîne : **Maquette** → **Détection** → **Arbre** → **Code**. La qualité du code final dépend presque entièrement de la qualité de l'arbre intermédiaire, qui reste l'étape la plus fragile de ce pipeline.

L'objectif du mémoire est de se concentrer exclusivement sur cette étape de construction de l'arbre, et de proposer une méthode hybride combinant précision géométrique et compréhension sémantique. Les livrables attendus sont un article de recherche et sa soutenance orale, prévue fin novembre.

2 Panorama complet des méthodes existantes

Avant de justifier notre choix méthodologique, il est nécessaire d'établir un panorama complet des familles de méthodes existantes pour la génération de code à partir d'images ou de maquettes UI. Ce panorama s'appuie sur les sections « travaux connexes » de plusieurs articles déjà étudiés, en particulier DesignCoder (Chen et al., 2025, Section 5) et Vu et al. (2025, arXiv:2504.18729).

2.1 Les familles identifiées

Famille	Exemples	Principe
Systèmes à base de règles	REMAUI (Nguyen & Csallner, 2015)	Assignation de composants UI à des modèles de code prédéfinis ; flexibilité limitée face à des designs nouveaux
Deep learning bout-en-bout	Pix2Code (Beltramelli, 2018), Sketch2Code (2019)	Un réseau de neurones convolutif transforme directement une capture d'écran en code, en un seul bloc
Apprentissage par renforcement	Soselia et al. (2023)	Traite la génération comme un problème de RL, avec le score IoU comme récompense
Prompting direct de gros VLM	Design2Code (Si et al., 2024)	Utilisation de GPT, Claude, Gemini sans fine-tuning spécifique
Hybride vision + deep learning	Bo et al. (2023)	Vision par ordinateur pour la structure, deep learning pour la génération de code
Guidage par structure hiérarchique	DeclarUI (Zhou et al., 2024), UICopilot (Gui et al., 2025)	L'organisation spatiale ou hiérarchique guide le modèle ou sa stratégie d'inférence
Représentation par graphe	Graph4GUI (Jiang et al., 2024)	Construction d'un graphe via CV + règles, puis réseau de neurones sur graphe ; objectif principal différent (compréhension, pas génération de code)

Table 1: Taxonomie des méthodes de génération UI-to-code, établie à partir des sections « travaux connexes » de DesignCoder et Vu et al.

2.2 Pourquoi une approche hybride mathématique + MLLM, et pas une autre famille

Notre choix s'est porté sur une approche hybride combinant calcul mathématique déterministe et modèle multimodal (MLLM), par élimination argumentée de chaque famille alternative :

- **Contrairement aux systèmes à base de règles** (REMAUI), qui exigent des métadonnées déjà connues et n'offrent aucune flexibilité face à des designs nouveaux, notre approche part

d'une image brute, sans métadonnée.

- **Contrairement au deep learning bout-en-bout** (Pix2Code, Sketch2Code), qui produit le code en une seule fois sans étape intermédiaire vérifiable, notre approche construit un arbre explicite et inspectable à chaque étape.
- **Contrairement au prompting direct de gros modèles multimodaux** (GPT, Claude, Gemini), qui hallucinent les positions exactes des éléments, notre approche garantit la géométrie par un calcul déterministe, jamais par une prédiction de l'IA.
- **Contrairement aux approches hybrides existantes** (DCGen, ScreenCoder, VSA), qui mélangent encore l'IA dans l'étape de structuration elle-même (voir Section 5 sur la nuance VSA), notre approche interdit strictement à l'IA de toucher à la position : elle n'intervient qu'après, uniquement pour nommer.

Synthèse : notre contribution n'est pas d'inventer une hybridation mathématique + IA — cette idée générale existe déjà sous diverses formes. Notre contribution précise est de rendre cette séparation **totale et irréversible** : zéro composant appris dans la construction géométrique, l'IA cantonnée à un unique rôle final de nommage, jamais de repositionnement.

3 L'ordre d'exécution : géométrie d'abord, sens ensuite

Une fois le principe hybride retenu, restait à déterminer l'ordre d'exécution. Deux ordres sont possibles : laisser l'IA structurer puis calculer la géométrie (comme DesignCoder), ou l'inverse (notre choix).

3.1 Preuve vérifiée : l'ordre de DesignCoder

Le texte source de DesignCoder (Section 2.1) précise :

> *« Group the UI elements based on their semantic, functional, and spatial relationships into UI components. Output a complete hierarchical tree of the grouped UI elements. »*

Ceci confirme que c'est le LLM qui construit l'arbre hiérarchique à partir du **sens**, la position (bbox) étant déjà connue via des métadonnées Figma extraites en amont — pas calculée par un algorithme mathématique pur.

3.2 Preuve vérifiée : l'ordre de ScreenCoder

Le texte source de ScreenCoder confirme, avec le même exemple cité dans l'article :

> *« the agent employs a Multimodal Large Language Model (MLLM) queried with prompts such as 'Where is the sidebar?' or 'Locate the header area.' The MLLM returns a set of grounded regions as bounding boxes and their associated labels. »*

Les auteurs assument même ce choix comme un avantage : *« Unlike traditional object detectors which require costly retraining, our MLLM-based approach is inherently extensible. »*

3.3 Notre choix : l'inverse, assumé

- Le calcul garantit une structure fiable et déterministe.
- L'IA n'a plus qu'un seul rôle simple : nommer, jamais positionner.
- Les erreurs de raisonnement du MLLM ne peuvent plus contaminer la géométrie.

- Nous faisons le choix inverse et assumé de ScreenCoder : sacrifier une partie de la flexibilité d'extension pour garantir un déterminisme total.

4 Les huit champs du nœud

Chaque champ du nœud est rattaché à l'une de quatre origines possibles : Neutre (structurel), Détecteur (position + type grossier), Géométrie (algorithme mathématique), ou MLLM (sens précis).

Champ	Origine	Justification et source externe
`id`	Neutre	L'algorithme de référence pour le Tree Edit Distance (Zhang & Shasha, 1989) nécessite une correspondance de nœuds entre deux arbres comparés. Sans identifiant unique, cette correspondance est impossible.
`bbox`	Détecteur	Position exacte fournie par le détecteur pour les feuilles, agrégée par calcul pour les nœuds internes. Champ fondateur, universel dans la littérature (UIED, Screen Parsing).
`type grossier`	Détecteur	Texte / Image / Composant. Consommé directement par l'algorithme Gestalt, qui distingue les widgets textuels des non-textuels pour former ses clusters (Xie et al., 2022, Section 3.1) — pas seulement un indice pour le MLLM.
`parent`	Géométrie	Référence au nœud englobant ; permet la remontée vers la racine et la fusion de groupes accidentellement séparés.
`children`	Géométrie	Liste **ordonnée**. Zhang & Shasha (1989) précisent qu'un arbre ordonné est un arbre dans lequel l'ordre gauche-droite entre nœuds frères est significatif — propriété formelle indispensable au calcul du Tree Edit Distance.
`type géométrique`	Géométrie	Feuille / Row / Column. Pilote l'algorithme de découpage (XY-Cut) et détermine la direction CSS générée (Prototype2Code, ReverseORC, DCGen).
`label sémantique`	MLLM	Contribution centrale. Preuve empirique de son importance : ablation de DesignCoder (Table 3) — sans regroupement sémantique (« w/o Grouping »), TreeBLEU chute de 0,66 à 0,38 et Container Match de 0,73 à 0,41. Sans lui, le code généré souffre d'une « surutilisation d'éléments `<div>` plutôt que des hiérarchies de composants structurées » (DesignCoder, Section 5.2, à propos de Prototype2Code).
`hash / empreinte`	Géométrie (post-construction)	Sérialisation de sous-arbres (technique classique de détection de doublons) et canonicalisation. VSA (arXiv:2512.20034) utilise une « découverte déterministe de motifs récurrents » via hachage et fusion de quasi-doublons pour un objectif directement comparable.

Table 2: Les huit champs du nœud, avec origine et justification externe vérifiée.

5 Nuance sur le vide de recherche identifié

Une recherche plus large que les six premiers articles étudiés a révélé que l'affirmation initiale « aucun travail existant ne sépare les deux rôles » était **trop forte**. Deux candidats sérieux ont été identifiés :

5.1 VSA (Visual-Structural Alignment), décembre 2025

Ce travail très récent (arXiv:2512.20034) sépare explicitement : (i) la reconstruction de structure hiérarchique, (ii) la découverte déterministe de motifs récurrents, (iii) la synthèse de code. Le

point (ii) est directement comparable à notre champ `hash`.

Nuance qui sauve notre originalité : leur étape (i) de reconstruction structurelle emploie *« a spatial-aware transformer »* — un réseau de neurones. Notre construction géométrique, elle, est **100% déterministe et non-apprise** (Gestalt + XY-Cut, zéro réseau de neurones dans cette étape).

5.2 ReDraw (Moran et al.)

Ce travail suit un ordre Détection → Classification → Assemblage, où la classification sémantique intervient **avant** l'assemblage en hiérarchie — l'ordre inverse du nôtre. Ce point mériterait une lecture complète du papier original pour confirmation détaillée, non encore réalisée à ce jour.

5.3 Reformulation de notre contribution

> *« Si certains travaux très récents (VSA, décembre 2025) séparent désormais les étapes de structuration et de synthèse, aucun ne garantit une construction géométrique 100% déterministe et non-apprise : VSA utilise encore un transformeur pour la reconstruction structurelle elle-même, ce qui réintroduit un risque d'erreur non-déterministe exactement là où nous voulons l'éliminer. »*

6 Justification du terrain web

Nos maquettes cibles sont des maquettes web. Cette section explique pourquoi, **sans comparaison directe au mobile** (faute de statistique de profondeur suffisamment précise et comparable pour le mobile — voir Section 8.3).

Nous utilisons l'outil Gestalt Grouping pour regrouper les éléments détectés en blocs naturels (cartes, listes, menus), sans avoir besoin d'espace vide entre eux. Cependant, d'après l'article qui le décrit (Xie et al., 2022, ESEC/FSE, arXiv:2206.10352), cet outil n'analyse que 2 à 3 niveaux de profondeur, et a été évalué principalement sur 1091 captures d'applications Android réelles (dataset Rico, 772 applications) ainsi que 20 prototypes Figma.

Cette limite est suffisante pour gérer des maquettes mobiles, généralement plus plates. Mais nos maquettes pouvant aussi être des maquettes web, qui présentent une profondeur DOM moyenne de 17 niveaux (minimum 2, maximum 40, mesurée sur 348 pages réelles — DCGen, Table 1), cette limite de 2-3 niveaux devient insuffisante.

Nous complétons donc Gestalt avec XY-Cut, appliqué récursivement à l'intérieur de chaque groupe déjà trouvé, pour continuer à établir les relations parent-enfant jusqu'à la vraie profondeur de l'arbre.

7 Le choix du détecteur

Notre besoin précis est un détecteur capable de fournir, pour chaque élément, sa position (bbox) et son type grossier (Texte/Image/Composant), en une seule passe et sans MLLM. Ces deux informations constituent le point de départ indispensable à la construction de l'arbre : c'est ensuite Gestalt et XY-Cut, appliqués sur cette liste plate d'éléments, qui détermineront qui est le parent de qui et, inversement, quels éléments sont les enfants de quel nœud.

7.1 Candidats écartés

Screen Parsing (Wu et al., UIST 2021, Faster R-CNN) : le texte source confirme, mot pour mot : *« We trained our models on two mobile UI datasets: (i) AMP, an internal dataset of*

~130,000 iOS screens, and (ii) RICO, a publicly available dataset of ~80,000 Android screens. » Aucune donnée web — écarté.

ImageCLEF DrawnUI 2021 : entraîné spécifiquement sur des captures de sites web réels (9630 captures), mais malgré une recherche approfondie, aucun poids ni code source publiquement accessible n'a été trouvé — inutilisable dans nos délais.

7.2 Détecteur retenu

UIED (Xie, Feng, Xing, Chen, Chen — ESEC/FSE 2020, pp. 1655–1659), intégré à l'outil Gestalt Grouping, fournit position et distinction texte/non-texte via une détection purement algorithmique (vision par ordinateur classique), sans réseau de neurones pour le raisonnement sémantique.

8 État d'avancement pratique — résultats non encore validés

> **Cette section décrit des tests en cours, pas des résultats validés.** La combinaison Gestalt + XY-Cut n'a pas encore été mesurée quantitativement.

8.1 Installation et déploiement

Test initial sur Google Colab (GPU T4 disponible gratuitement), puis migration en local via WSL (Windows Subsystem for Linux) suite à un blocage de sécurité Windows (Smart App Control) empêchant le chargement des bibliothèques Python compilées (pandas).

8.2 Corrections nécessaires

Deux incompatibilités de code ont été identifiées et corrigées, dues à l'évolution des bibliothèques Python depuis la publication de l'outil (2022) :

- usage de `time.clock()`, supprimé depuis Python 3.8, remplacé par `time.time()` ;
- gestion défaillante de l'absence de détection de texte lorsque l'OCR est désactivé (`is_ocr=False`), corrigée par une vérification d'existence du fichier avant chargement.

8.3 Tests menés

- **Maquette mobile** (application de VTC) : la détection brute (`detect_element`) a correctement délimité les cartes et boutons, validant le fonctionnement de base de l'outil. Le texte apparaît fragmenté par couleur, ce qui est attendu en l'absence d'OCR.
- **Capture web** : une première capture a révélé un biais méthodologique — la capture incluait la fenêtre du navigateur et la barre des tâches Windows, polluant la détection. Une capture de page complète (via les outils de développement du navigateur) est nécessaire pour la suite.

8.4 Limite non résolue à ce jour

Nous n'avons pas trouvé de statistique de profondeur moyenne pour le mobile directement comparable au chiffre de 17 niveaux mesuré pour le web (DCGen). Une étude sur le dataset RICO indique qu'environ 39% des feuilles se situent au premier niveau des arbres, suggérant des hiérarchies plus plates, mais sans moyenne chiffrée équivalente. Ce point reste une limite honnête de l'argumentaire, à mentionner si interrogé.

8.5 Prochaines étapes immédiates

1. Refaire les tests sur de vraies captures de pages web complètes (sans navigateur ni barre des tâches).
2. Confirmer l'effet de `recognize_layout()` (l'étape Gestalt) sur la fragmentation du texte observée.
3. Écrire l'algorithme XY-Cut nous-mêmes (absent de l'outil téléchargé), à appliquer récursivement à l'intérieur de chaque groupe Gestalt.
4. Mesurer quantitativement (Tree Edit Distance, Container Match, TreeBLEU) une fois le pipeline complet fonctionnel.

9 Feuille de route générale

1. Cadrage du sujet — terminé
2. Choix de la stratégie de détection — terminé
3. Définition formelle de la structure de l'arbre — terminé (8 champs)
4. **Construction de l'algorithme géométrique — en cours, à l'essai**
5. Intégration de l'étiquetage sémantique
6. Préparation des données de test
7. Mesure des résultats
8. Rédaction de l'article
9. Relecture et finalisation

10 Bibliographie

[1] Zhou, T. et al. *Bridging design and development with automated declarative UI code generation.* arXiv:2409.11667, 2024.

[2] Chen, Y. et al. *DesignCoder: Hierarchy-Aware and Self-Correcting UI Code Generation with Large Language Models.* arXiv:2506.13663, 2025.

[3] Jiang, Y., Stuerzlinger, W., Lutteroth, C. *ReverseORC: Reverse Engineering of Resizable UI Layouts with OR-Constraints.* CHI 2021.

[4] Wu, J., Zhang, X., Nichols, J., Bigham, J.P. *Screen Parsing: Towards Reverse Engineering of UI Models from Screenshots.* UIST 2021. arXiv:2109.08763.

[5] Vu, T.D., Hoang, C., Hy, T.S. *Multimodal Graph Representation Learning for Website Generation Based on Visual Sketch.* arXiv:2504.18729.

[6] Jiang, Y. et al. *ScreenCoder: Advancing Visual-to-Code Generation via Modular Multimodal Agents.* arXiv:2507.22827.

[7] Xie, M., Xing, Z., Feng, S., Xu, X., Zhu, L., Chen, C. *Psychologically-Inspired, Unsupervised Inference of Perceptual Groups of GUI Widgets from GUI Images.* ESEC/FSE 2022, pp. 332–343. arXiv:2206.10352.

[8] Xie, M., Feng, S., Xing, Z., Chen, J., Chen, C. *UIED: a hybrid tool for GUI element detection.* ESEC/FSE 2020, pp. 1655–1659.

[9] Gui, Y. et al. *UICopilot: Automating UI synthesis via hierarchical code generation from webpage designs.* The Web Conference 2025.

[10] Zhang, K., Shasha, D. *Simple fast algorithms for the editing distance between trees and related problems.* SIAM Journal on Computing, 1989.

[11] VSA. *Visual-Structural Alignment.* arXiv:2512.20034, 2025.

[12] Moran, K. et al. *ReDraw: Machine Learning-Based Prototyping of Graphical User Interfaces for Mobile Apps.* IEEE TSE, 2018.